Mise en place d’un Firewall sous Ubuntu 22.04 avec iptables

🎯 Objectif du TP

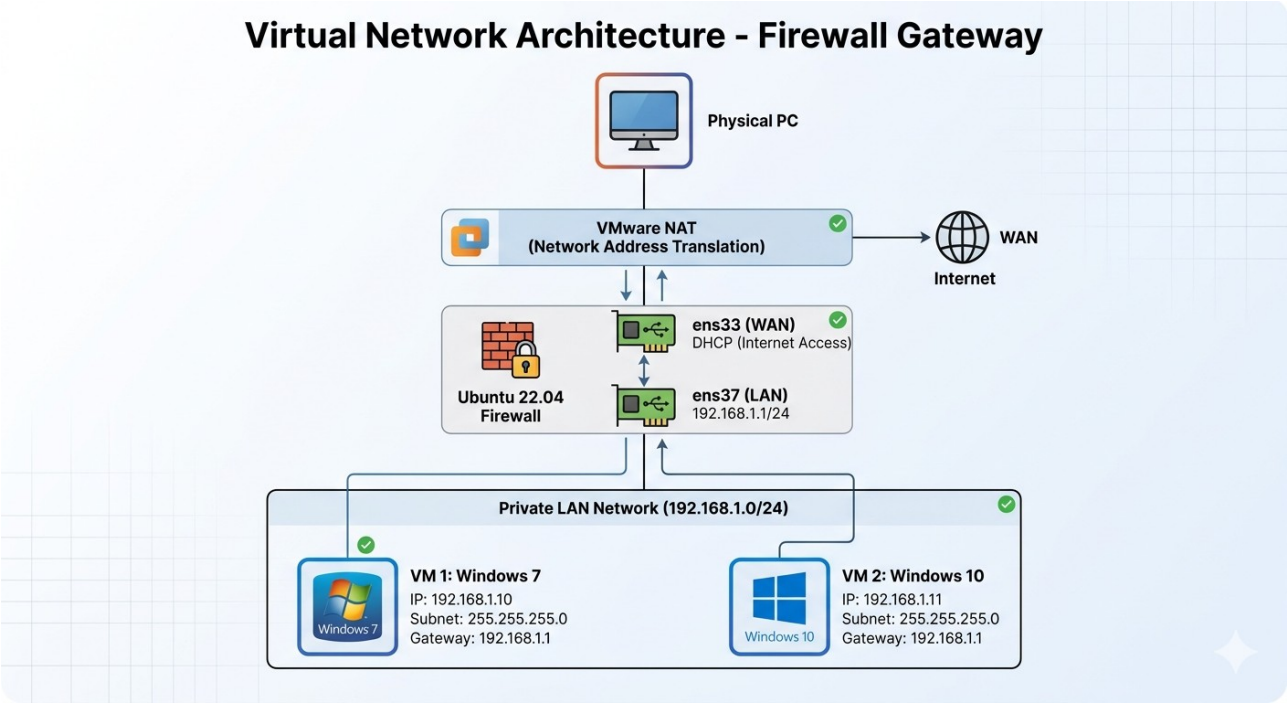
Ce travail pratique a pour objectif de :

- Mettre en place un **pare-feu Linux avec iptables**
- Configurer un **routage sécurisé entre un réseau LAN et Internet**
- Implémenter le **NAT (Network Address Translation)**
- Appliquer des **règles de filtrage strictes (DROP par défaut)**

🌐 Architecture réseau

- 1 machine Ubuntu version 22 jouant le rôle de **Firewall / Routeur**
- 2 machines clientes (**Windows 7 et Windows 10**)
- Réseau structuré en :
  - **WAN (Internet)** via NAT VMware
  - **LAN (192.168.1.0/24)**

Architecture



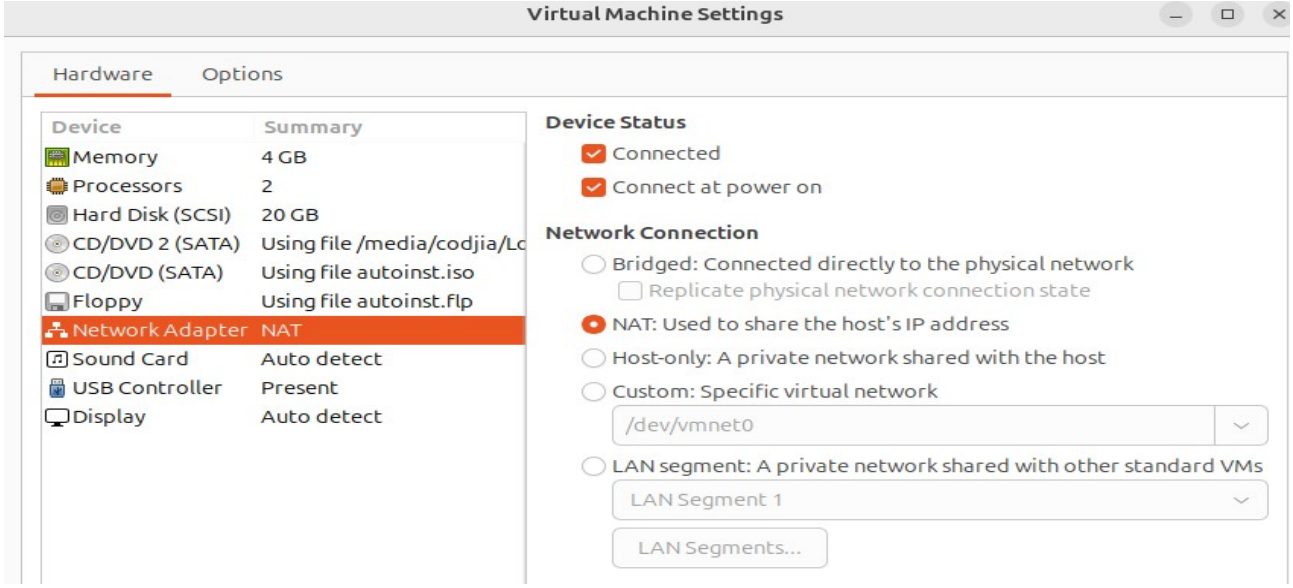
👉 Interfaces :

- `ens33` : WAN (DHCP) en Nat
- `ens37` : LAN (192.168.1.1)

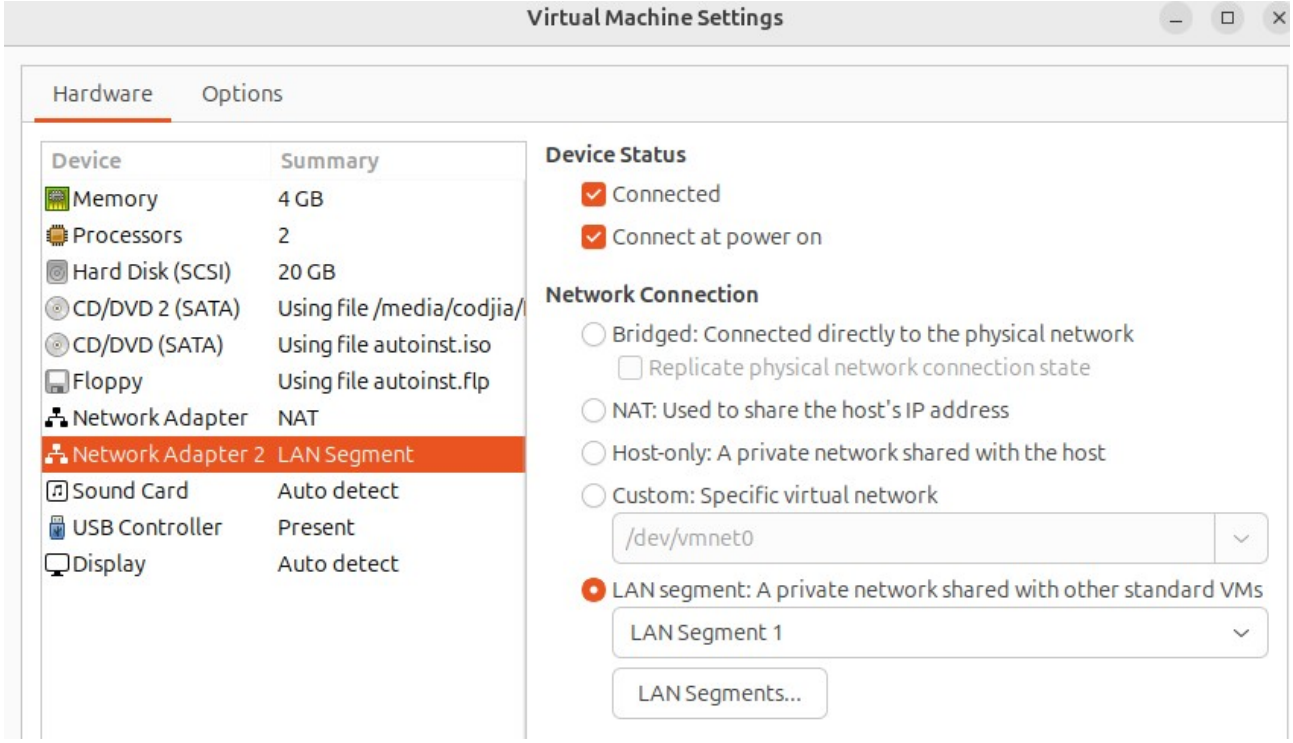
⚙️ Étapes de mise en œuvre

🔹 1. Identification des interfaces

- Prémier carte réseau activé pour le firewall en Nat donc il va utiliser l'adresse physique de la machine hôte.

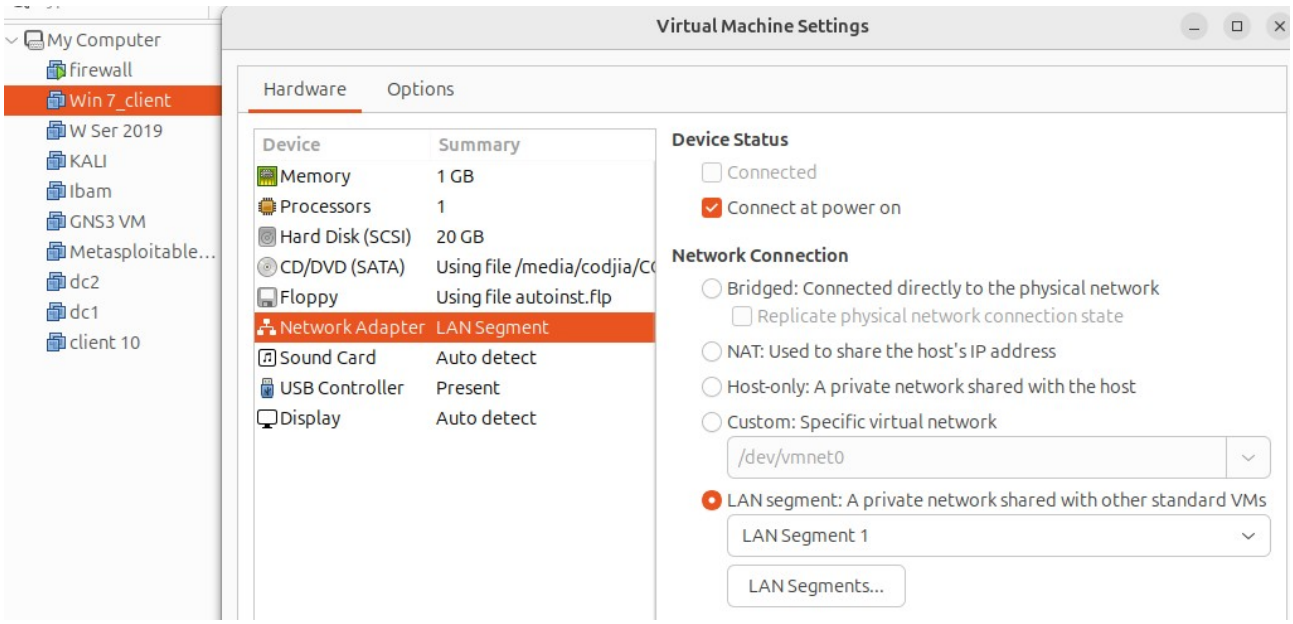


- Deuxième carte réseau ajouté que j'ai appélé Lan segment 1

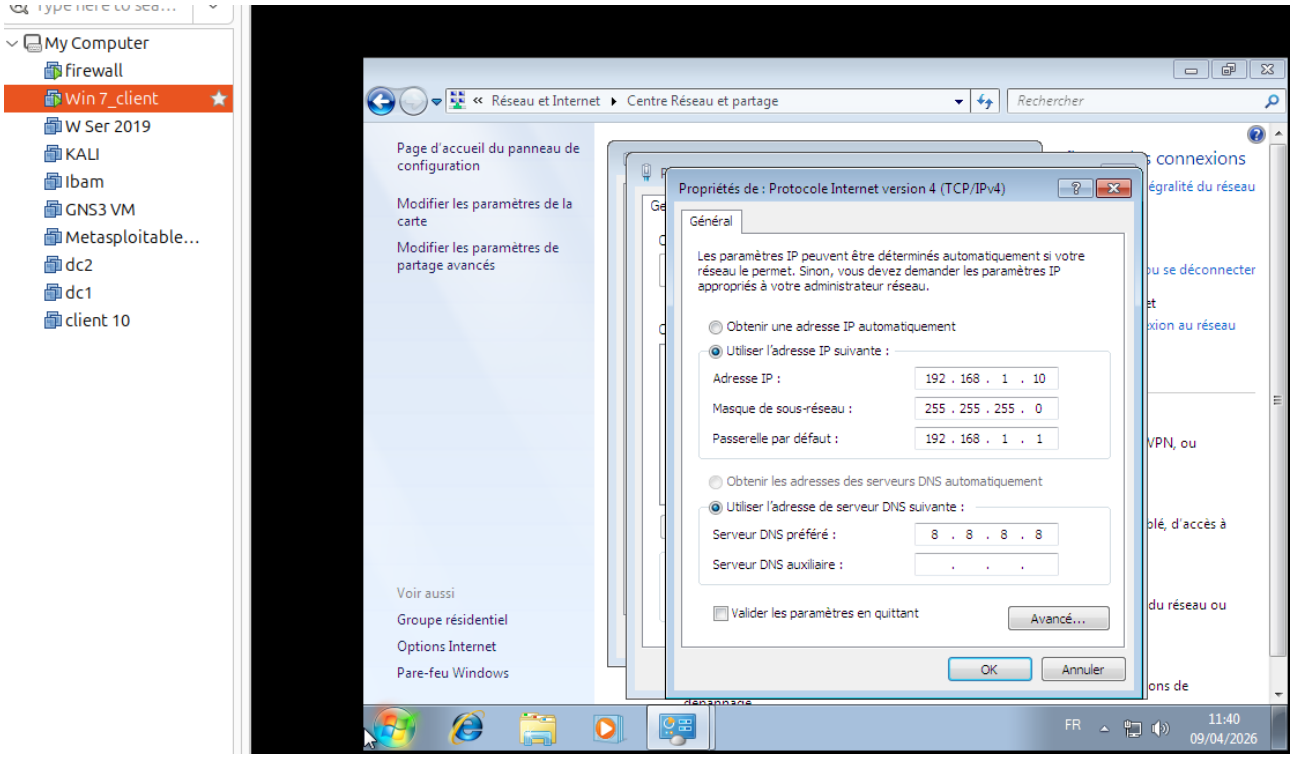


- Nous allons configuré rapidement nos machines clients et les ajouter dans le réseau LAN segment 1

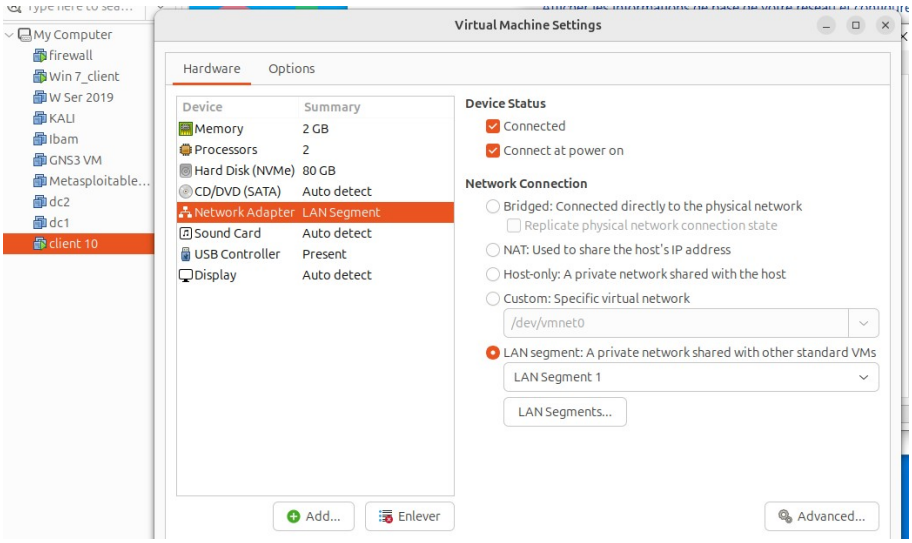
Pour la machine client 7



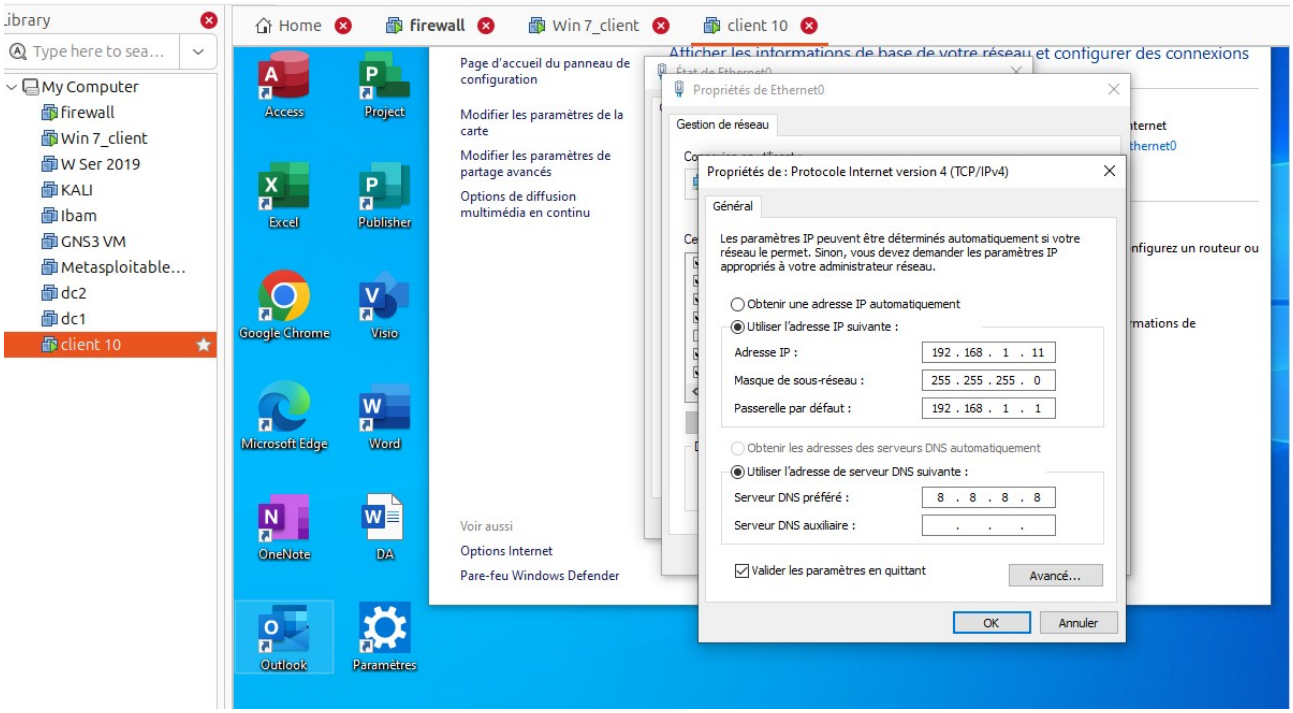
Configuration de l'adresse IP sur le client 7



Pour la machine client 10



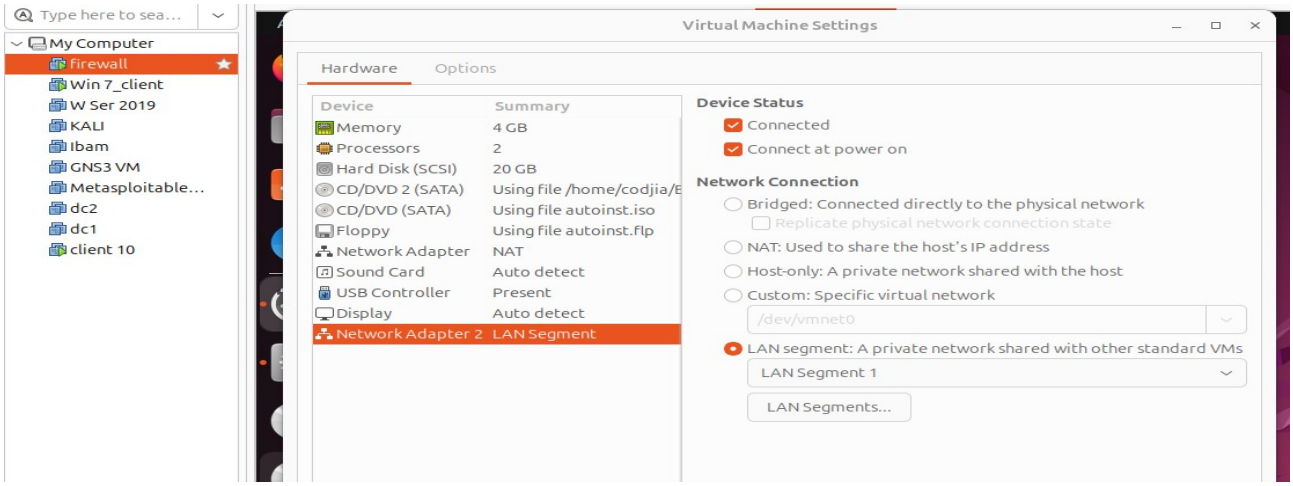
Configuration de l'adresse IP sur le client 10



Après cela, nos machines clients n'auront pas accès à internet mais les ping avec l'adresse local de firewall doivent marcher 192.168.1.1

Pour notre Machine linux (Firewall)

1. Ajoutons une carte réseau LAN et Utilisons aussi le NAT

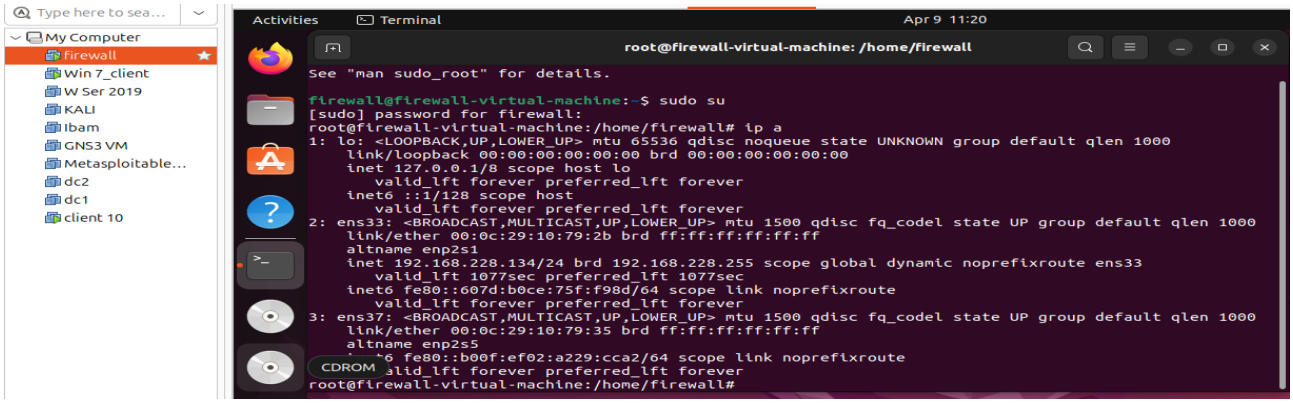


Passons au configurations du firewall

Commande utilisée :

ip a

Permet d'identifier les interfaces réseau actives et leurs adresses IP.



📝 **Description :** Affiche toutes les interfaces réseau avec leurs adresses IP, état (UP/DOWN) et caractéristiques.

🖥️ Exemple de sortie attendue :

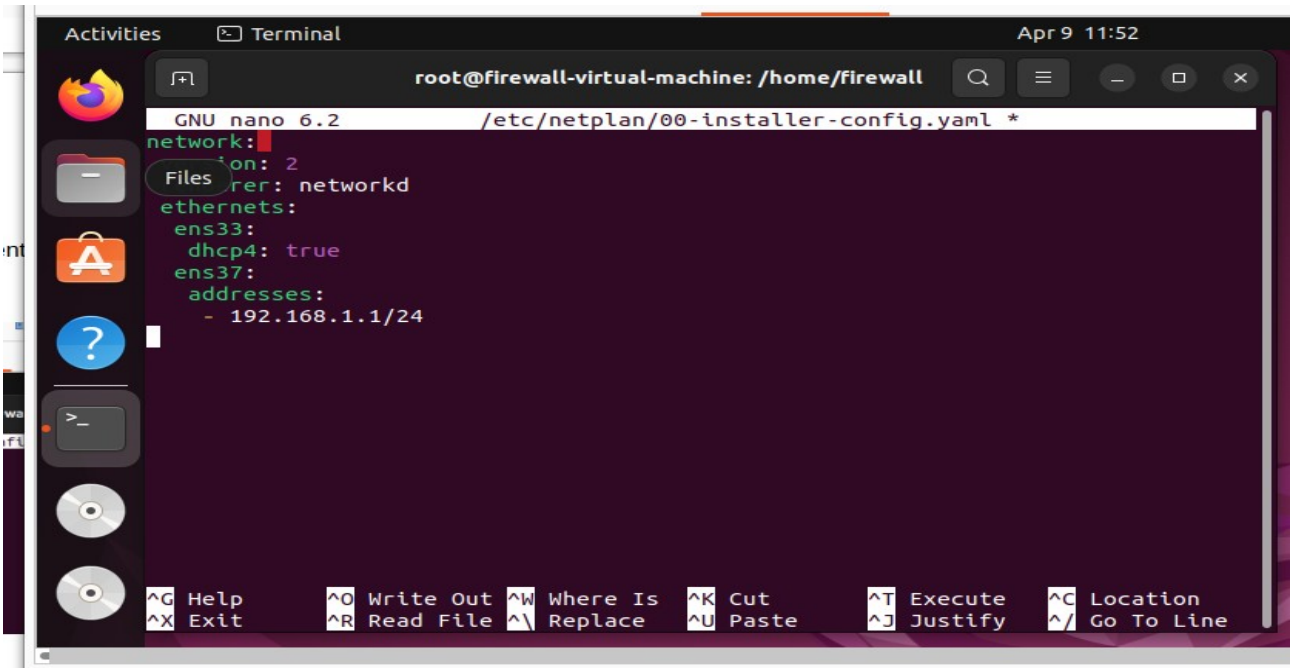
text

2: ens33: <BROADCAST,MULTICAST,UP> mtu 1500
    inet 192.168.xxx.xxx/24 ...
3: ens37: <BROADCAST,MULTICAST,UP> mtu 1500
    (pas d'IP encore)

2. Configuration réseau (Netplan)

- Activation DHCP côté WAN (NAT)
- Attribution d'une IP statique au LAN manuellement

sudo nano /etc/netplan/00-installer-config.yaml



Application :

Applique la configuration sans redémarrer. Persistante après reboot.

`sudo netplan apply`

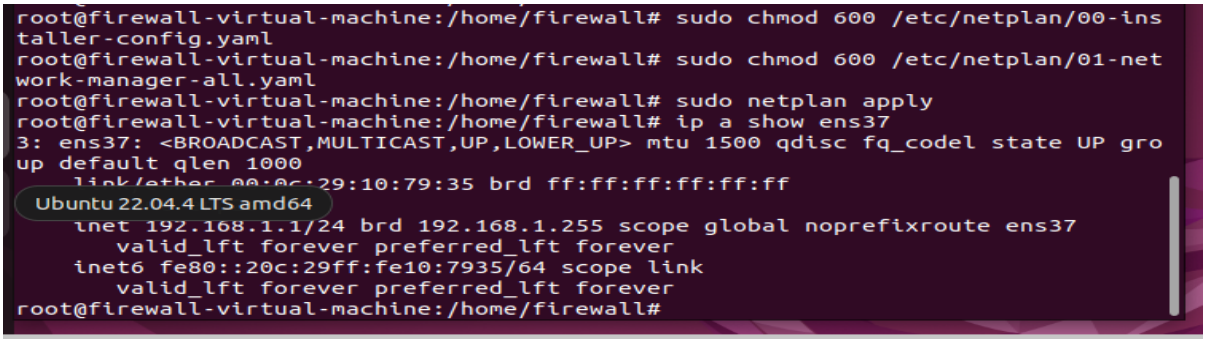
⚠️ Netplan exige que ces fichiers soient **protégés (lecture root uniquement)**, sinon il affiche ces warnings. Donc on dois taper :

sudo chmod 600 /etc/netplan/00-installer-config.yaml

sudo chmod 600 /etc/netplan/01-network-manager-all.yaml

Vérification

ip a show ens37



3. Installation des outils

Mise à jour des dépôts :

sudo apt update

Actualise la liste des paquets disponibles depuis les dépôts Ubuntu.

3. Installation d'iptables-persistent :

sudo apt install -y iptables-persistent

`Répond "oui" automatiquement`

•iptables-persistent : sauvegarde et restaure les règles au redémarrage

•Les règles sont stockées dans `/etc/iptables/rules.v4`

4. Activation du routage IP (IP Forward)

Activation permanente :

Sudo nano /etc/sysctl.conf

Ajoute cette ligne ou décomenter :

Net.ipv4.ip_forward=1

Appliquer

Sudo sysctl -p

Vérification finale

`cat /proc/sys/net/ipv4/ip_forward`

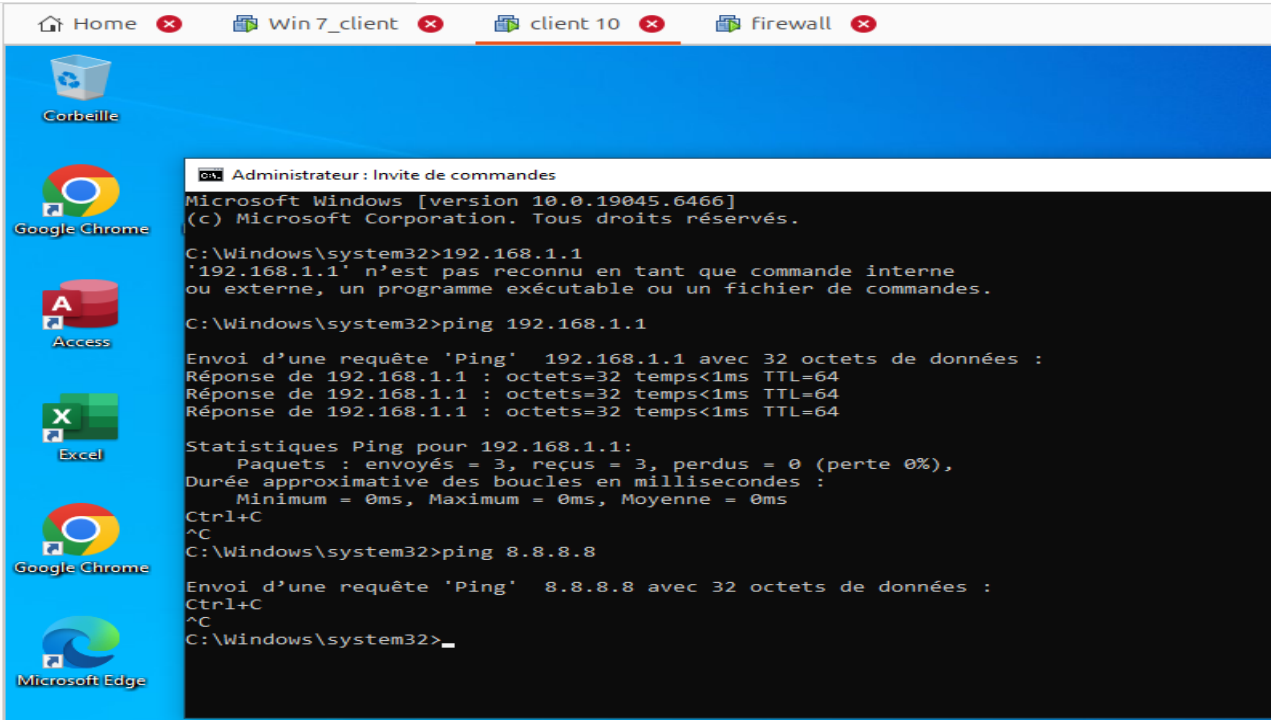
Résultat attendu :

1 ce qui signifie que le routage est activé

```
root@firewall-virtual-machine:/home/firewall# cat /proc/sys/net/ipv4/ip_forward
1
root@firewall-virtual-machine:/home/firewall#
```

```
root@firewall-virtual-machine:/home/firewall# sudo sysctl -p
net.ipv4.ip_forward = 1
root@firewall-virtual-machine:/home/firewall# cat /proc/sys/net/ipv4/ip_forward
1
root@firewall-virtual-machine:/home/firewall# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=113 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=86.2 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=55.1 ms
^C
--- 8.8.8.8 ping statistics ---
```

Actuellement les machines virtuelles peuvent ping l'ip du firewall mais ne peut pas aller sur l'internet



- **5. Configuration du firewall (iptables)**

⚠️ Étape 5.1 : Réinitialisation complète

sudo iptables -F (Supprime toutes les règles)
sudo iptables -t nat -F (Vide les règles de traduction d'adresses)
sudo iptables -X (Nettoie les chaînes vides)

```
root@firewall-virtual-machine:/home/firewall# sudo iptables -F
root@firewall-virtual-machine:/home/firewall# sudo iptables -t nat -F
root@firewall-virtual-machine:/home/firewall# sudo iptables -X
root@firewall-virtual-machine:/home/firewall#
```

⚠️ Étape 5.2 : Politique par défaut (DROP)

🔐 **Politique de sécurité stricte**

- INPUT : DROP
- OUTPUT : DROP
- FORWARD : DROP

sudo iptables -P INPUT DROP (Définit la politique par défaut, Tout paquet entrant est bloqué par défaut))
sudo iptables -P OUTPUT DROP (Tout paquet sortant est bloqué par défaut
sudo iptables -P FORWARD DROP (Tout paquet sortant est bloqué par défaut)

👉 Principe : **tout bloquer, puis autoriser uniquement le nécessaire**

```
root@firewall-virtual-machine:/home/firewall# sudo iptables -P INPUT DROP
root@firewall-virtual-machine:/home/firewall# sudo iptables -P OUTPUT DROP
root@firewall-virtual-machine:/home/firewall# sudo iptables -P FORWARD DROP
root@firewall-virtual-machine:/home/firewall#
```

```
root@firewall-virtual-machine:/home/firewall# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1061ms

root@firewall-virtual-machine:/home/firewall# ping 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
Ubuntu 22.04.4 LTS amd64
--- 127.0.0.1 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1021ms

root@firewall-virtual-machine:/home/firewall#
```

Vérification

Étape 5.3 : Boucle locale (loopback)

on autorise uniquement ce qui est nécessaire.

sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT

explication

-A (Ajoute une règle à la fin de la chaîne**)**

-i lo (Interface loopback (127.0.0.1)

-o lo (Interface loopback**)**

-j ACCEPT (Accepte le paquet**)**

```
root@firewall-virtual-machine:/home/firewall# sudo iptables -A INPUT -i lo -j ACCEPT
root@firewall-virtual-machine:/home/firewall# sudo iptables -A OUTPUT -o lo -j ACCEPT
root@firewall-virtual-machine:/home/firewall#
```

Résultat

```
root@firewall-virtual-machine:/home/firewall# ping 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.207 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.063 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.060 ms
^C
--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2030ms
rtt min/avg/max/mdev = 0.060/0.110/0.207/0.068 ms
root@firewall-virtual-machine:/home/firewall# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2088ms

root@firewall-virtual-machine:/home/firewall#
```

Étape 5.4 : Connexions établies et associées

Autoriser les réponses aux connexions déjà autorisées (évite de bloquer le retour).

sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

sudo iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

Explication des commandes

-m conntrack (Utilise le module de suivi de connexion)

--ctstate (État de la connexion)

ESTABLISHED (Connexion déjà établie (réponses))

RELATED (Connexion liée à une existante (ex: FTP data))

Étape 5.5 : Accès au réseau local (LAN)

sudo iptables -A INPUT -i ens37 -j ACCEPT

sudo iptables -A OUTPUT -o ens37 -j ACCEPT

Objectif : Permettre au firewall de communiquer avec les machines du LAN (192.168.1.10 et 192,168,1,11).

```
root@firewall-virtual-machine:/home/firewall# ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=128 time=0.669 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=128 time=0.709 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=128 time=0.620 ms
^C
--- 192.168.1.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2083ms
rtt min/avg/max/mdev = 0.620/0.666/0.709/0.036 ms
root@firewall-virtual-machine:/home/firewall# ping 192.168.1.11
PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
64 bytes from 192.168.1.11: icmp_seq=1 ttl=128 time=0.559 ms
om 192.168.1.11: icmp_seq=2 ttl=128 time=0.792 ms
Floppy Disk
--- 192.168.1.11 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1015ms
rtt min/avg/max/mdev = 0.559/0.675/0.792/0.116 ms
root@firewall-virtual-machine:/home/firewall#
```

```
root@firewall-virtual-machine:/home/firewall# ping www.google.com



^C
root@firewall-virtual-machine:/home/firewall#
```

Étape 5.6 : DNS (résolution de noms)

Permettre au firewall de résoudre des noms de domaine (ex: google.com → IP).

sudo iptables -A OUTPUT -p udp --dport 53 -o ens33 -j ACCEPT

Option Explication -p udp Protocole UDP --dport 53 Port destination 53 (DNS) -o ens33 Sortie par l'interface WAN

```
root@firewall-virtual-machine:/home/firewall# sudo iptables -A OUTPUT -p udp --dport 53 -o ens33 -j ACCEPT
root@firewall-virtual-machine:/home/firewall# ping www.google.com
PING www.google.com (142.251.152.119) 56(84) bytes of data.
^C
--- www.google.com ping statistics ---
23 packets transmitted, 0 received, 100% packet loss, time 22553ms

root@firewall-virtual-machine:/home/firewall# ping 8.8.8.8
8 (8.8.8.8) 56(84) bytes of data.
Floppy Disk
--- 8.8.8.8 ping statistics ---
```

Étape 5.7 : Autoriser tout ICMP (plus simple, pour tester)

sudo iptables -A INPUT -p icmp -j ACCEPT

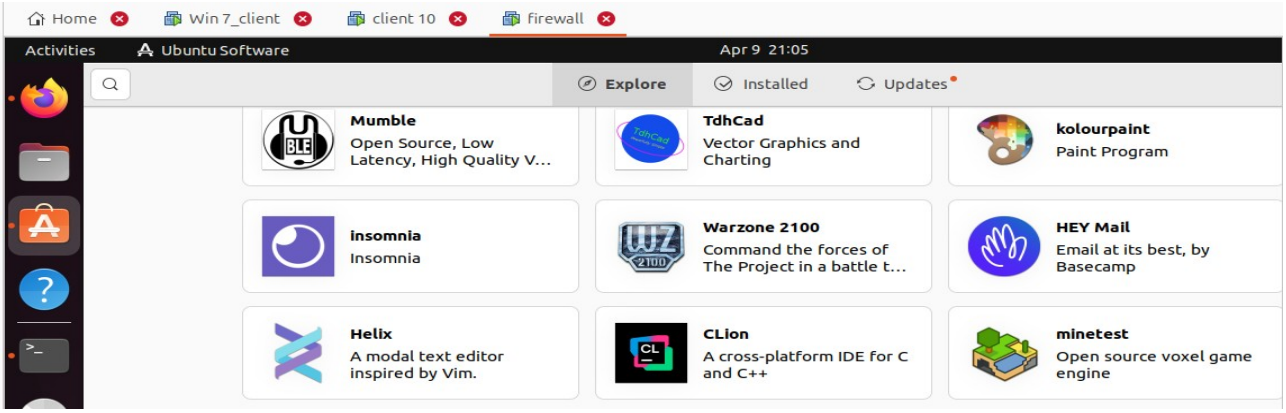
sudo iptables -A OUTPUT -p icmp -j ACCEPT

```
root@firewall-virtual-machine:/home/firewall# sudo iptables -A INPUT -p icmp -j ACCEPT
root@firewall-virtual-machine:/home/firewall# sudo iptables -A OUTPUT -p icmp -j ACCEPT
root@firewall-virtual-machine:/home/firewall# ping www.google.com
PING www.google.com (142.251.154.119) 56(84) bytes of data.
64 bytes from 142.251.154.119 (142.251.154.119): icmp_seq=1 ttl=128 time=39.9 ms
Ubuntu 22.04.4 LTS amd64 154.119 (142.251.154.119): icmp_seq=2 ttl=128 time=41.9 ms
^C
--- www.google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 39.863/40.889/41.915/1.026 ms
root@firewall-virtual-machine:/home/firewall# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=36.9 ms
^C
--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 36.918/36.918/36.918/0.000 ms
root@firewall-virtual-machine:/home/firewall#
```

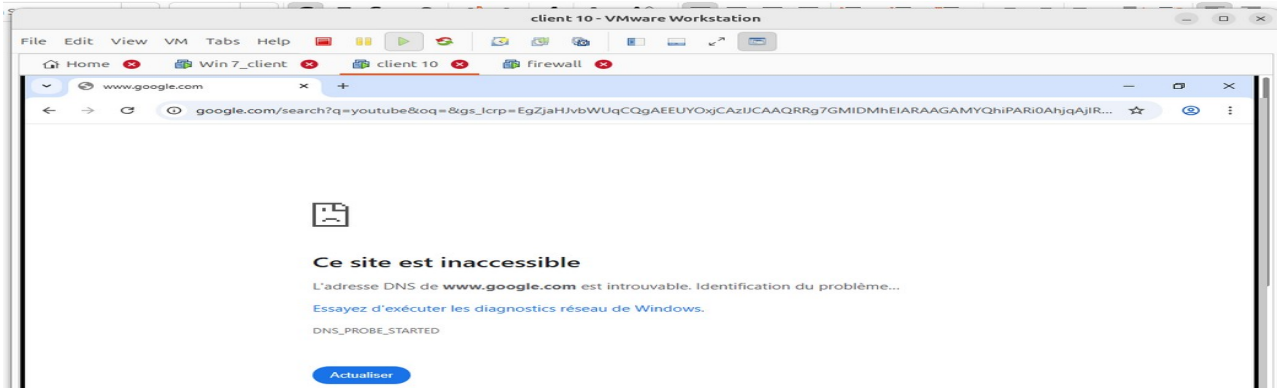
Étape 5.8 : Accès Internet depuis le firewall

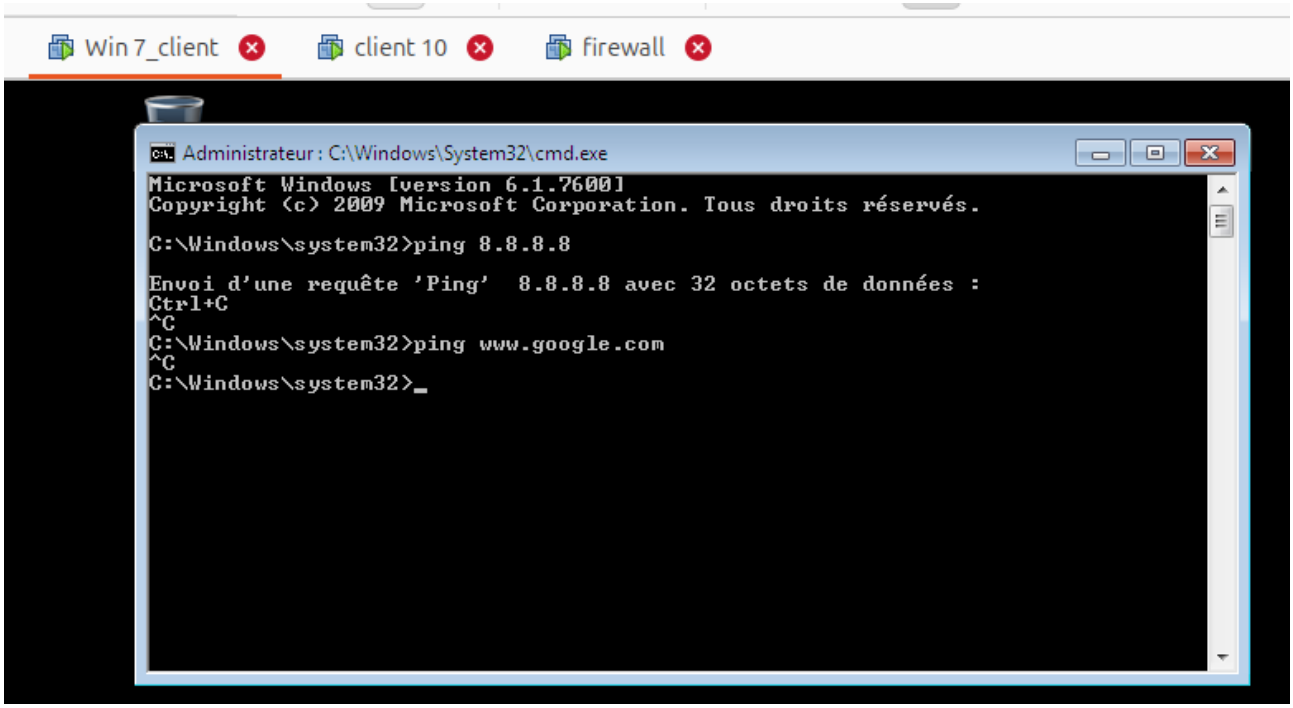
Permettre au firewall lui-même de naviguer, faire des mises à jour, etc.

sudo iptables -A OUTPUT -o ens33 -j ACCEPT



Pour l'instant les machines virtuelle n'ont pas acces à l'internet





Étape 5.9 : Transfert LAN → WAN (les clients accèdent à Internet)

Autoriser les machines du LAN (192.168.1.10, .11) à ouvrir des connexions vers Internet.

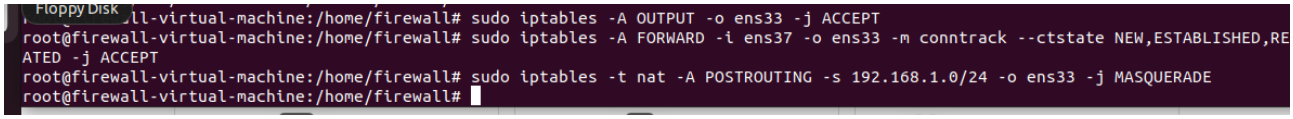
sudo iptables -A FORWARD -i ens37 -o ens33 -m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT

Autoriser les machines du réseau local (ens37) à initier et maintenir des connexions vers Internet (ens33), tout en laissant passer les réponses.

Étape 5.10 : NAT (Masquerade)

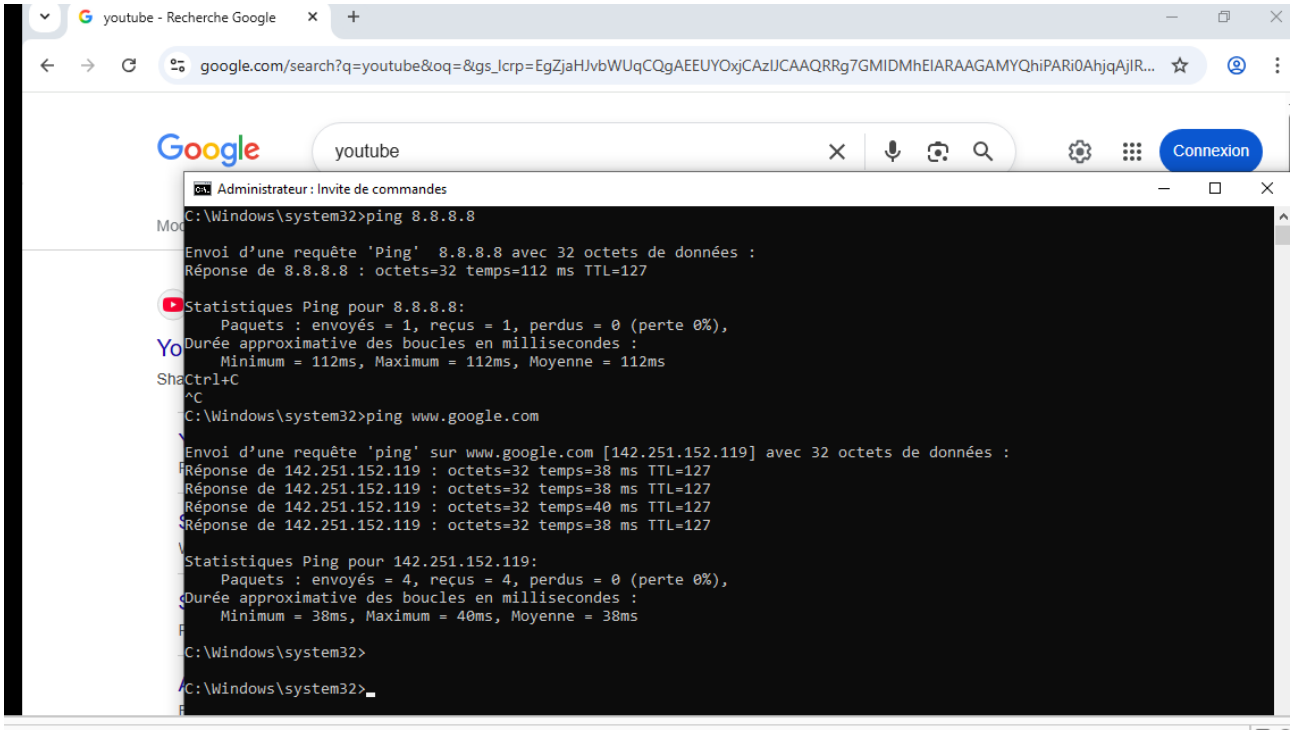
sudo iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o ens33 -j MASQUERADE

Permettre aux machines du réseau local (192.168.1.x) d'accéder à Internet en utilisant l'adresse IP publique du firewall.

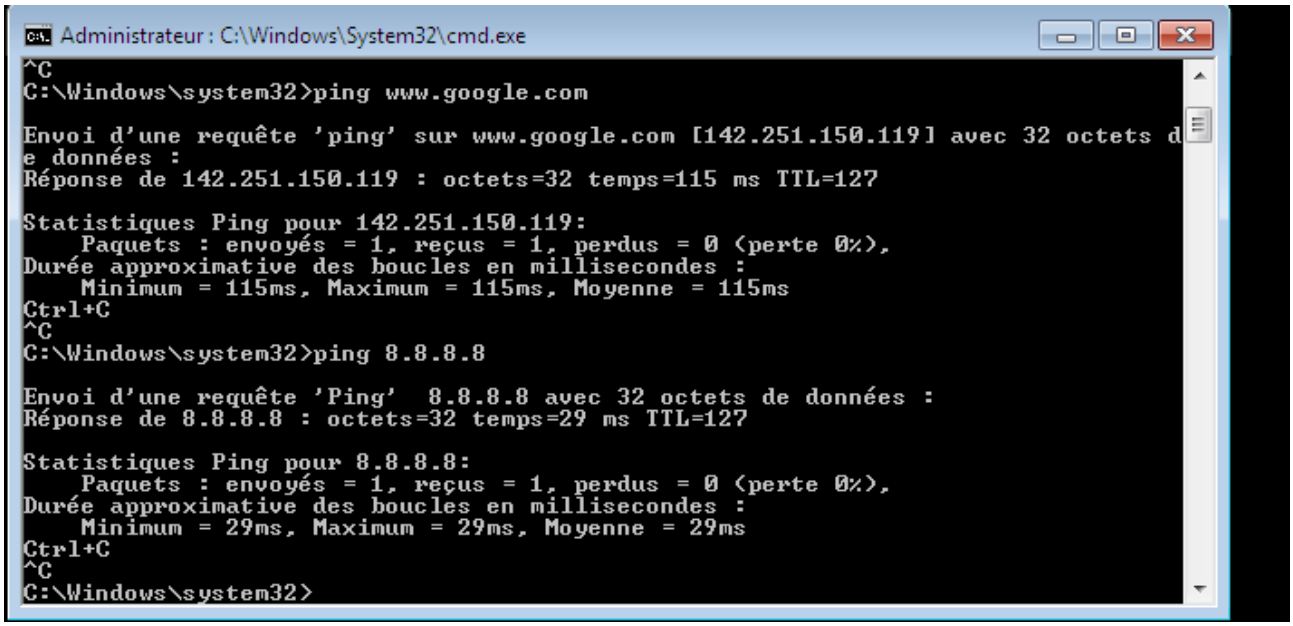


Résultat

-pour le client 10



-pour le client 7



5.11 : Journalisation (logging) – option avancée

Tracer les paquets destinés au firewall à des fins de débogage ou de surveillance (détection de tentatives d'intrusion).

sudo iptables -A INPUT -j LOG --log-prefix "DROP_INPUT: "

```
apt install sudo
root@firewall-virtual-machine:/home/firewall# sudo iptables -A INPUT -j LOG --log-prefix "DROP_INPUT:"
CDROM
root@firewall-virtual-machine:/home/firewall# sudo iptables -L INPUT -n -v
Chain INPUT (policy DROP 90 packets, 10850 bytes)
 pkts bytes target     prot opt in     out     source               destination
 3661  355K ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
 5327   22M ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
   50  5420 ACCEPT     all  --  ens37  *       0.0.0.0/0            0.0.0.0/0
    0     0 ACCEPT     icmp --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 LOG        all  --  *      *       0.0.0.0/0            0.0.0.0/0            LOG flags 0 level 4 prefix "DROP_INPUT
root@firewall-virtual-machine:/home/firewall#
```

6. Sauvegarde des règles (persistance)

Garantir que l'ensemble des règles iptables configurées

soient automatiquement restaurées après chaque redémarrage du système, assurant ainsi une persistance complète du pare-feu.

sudo iptables-save | sudo tee /etc/iptables/rules.v4 > /dev/null

7. Vérifications finales

Afficher les règles de filtrage :

sudo iptables -L -n -v

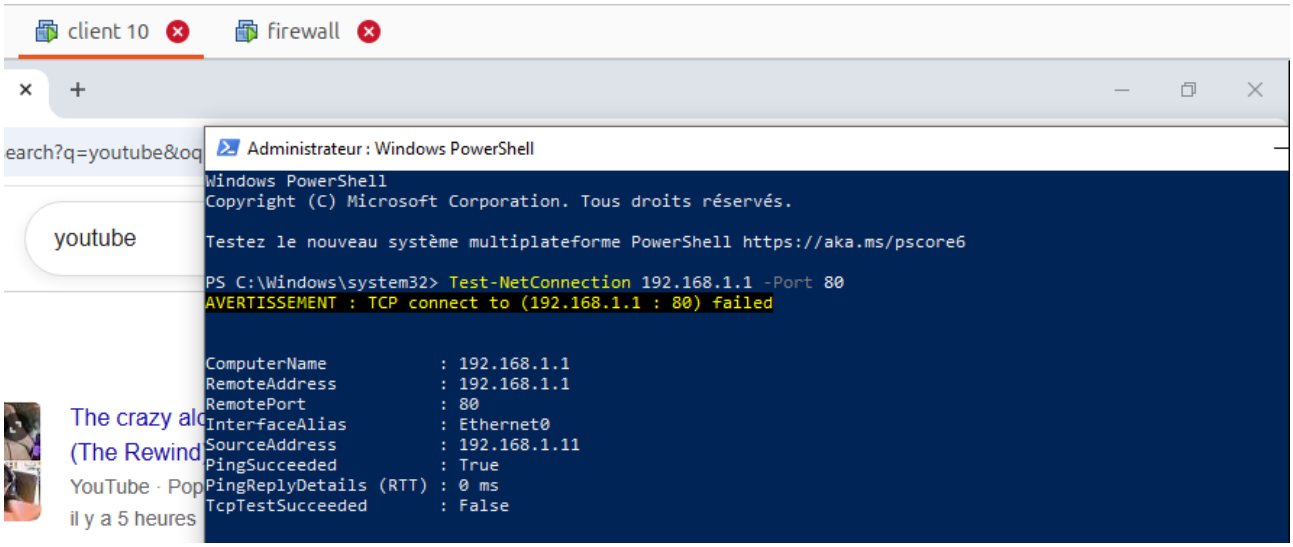
```
root@firewall-virtual-machine:/home/firewall# sudo iptables -L INPUT -n -v
Chain INPUT (policy DROP 90 packets, 10850 bytes)
 pkts bytes target     prot opt in     out     source               destination
 3661  355K ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
 5327   22M ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
   50  5420 ACCEPT     all  --  ens37  *       0.0.0.0/0            0.0.0.0/0
    0     0 ACCEPT     icmp --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 LOG        all  --  *      *       0.0.0.0/0            0.0.0.0/0            LOG flags 0 level 4 prefix "DROP_INPUT:"
root@firewall-virtual-machine:/home/firewall#
root@firewall-virtual-machine:/home/firewall# sudo iptables-save | sudo tee /etc/iptables/rules.v4 > /dev/null
root@firewall-virtual-machine:/home/firewall# sudo iptables -L -n -v
Chain INPUT (policy DROP 92 packets, 11069 bytes)
 pkts bytes target     prot opt in     out     source               destination
 4042  402K ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
 7827   25M ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
   54  5883 ACCEPT     all  --  ens37  *       0.0.0.0/0            0.0.0.0/0
    0     0 ACCEPT     icmp --  *      *       0.0.0.0/0            0.0.0.0/0
    2   219 LOG        all  --  *      *       0.0.0.0/0            0.0.0.0/0            LOG flags 0 level 4 prefix "DROP_INPUT:"

Chain FORWARD (policy DROP 5033 packets, 350K bytes)
 pkts bytes target     prot opt in     out     source               destination
```

8. A partir de la machine cliente, faites un nmap sous le port 80. Résultat ?

cmd

nmap -p 80 192.168.1.1



Interprétation :

•`TcpTestSucceeded : False` → Le port 80 est **FERMÉ ou FILTRÉ** par le firewall

`Conclusion :` `Le port 80 est filtered. Aucun serveur HTTP n'est installé sur le firewall et aucune règle iptables n'autorise le trafic entrant sur ce port.`

9-Pour tester tous les ports

powershell

```
80,443,22,21,25,3306 | ForEach-Object {
   $result = Test-NetConnection 192.168.1.1 -Port $_ -WarningAction SilentlyContinue
   if ($result.TcpTestSucceeded) {
      Write-Host "Port $_ : OPEN" -ForegroundColor Green
   } else {
      Write-Host "Port $_ : FILTERED ou CLOSED" -ForegroundColor Red
   }
```

}

Conclusion

Ce travail pratique m'a permis de maîtriser la configuration d'un **firewall Linux** avec **iptables**, outil fondamental de la sécurité réseau. J'ai mis en place une machine Ubuntu 22.04 jouant le rôle de pare-feu entre un réseau local (LAN) et Internet (WAN).

Les principaux objectifs atteints sont :

•La **politique de sécurité DROP par défaut** bloquant tout trafic non explicitement autorisé
•L'**activation du routage IP** (IP forward) pour permettre le transfert des paquets entre interfaces
•Le **filtrage entrant, sortant et transféré** via les chaînes INPUT, OUTPUT et FORWARD
•La **traduction d'adresses (NAT/MASQUERADE)** pour permettre aux clients du LAN d'accéder à Internet avec une seule adresse IP publique
•La **persistance des règles** après redémarrage grâce à iptables-persistent
•La **journalisation des paquets bloqués** pour faciliter le débogage

Les tests de validation (ping, navigation web, scan nmap) ont confirmé le bon fonctionnement du pare-feu. Les clients Windows 10 du réseau local accèdent parfaitement à Internet, tandis que le firewall reste invisible depuis l'extérieur grâce à la politique restrictive.

Ce TP constitue une **base solide** pour l'administration système et la sécurité réseau. Les compétences acquises sont directement transposables en entreprise pour la sécurisation de serveurs ou de passerelles Internet.

12.Explication

Le **masquerading** (ou **MASQUERADE**) est une forme de **NAT (Network Address Translation)**. Il remplace l'adresse IP source **privée** (ex: `192.168.1.10`) par l'adresse IP **publique** de l'interface de sortie (ex: `192.168.228.134`).

Pourquoi c'est nécessaire ?

•Les adresses privées (192.168.x.x, 10.x.x.x, 172.16.x.x) ne sont **pas routables sur Internet**
•Sans NAT, les paquets du client Windows arriveraient sur Internet avec une adresse source `192.168.1.10` → ils seraient **rejetés**
•Le NAT permet à **plusieurs machines** de partager **une seule adresse IP publique**

🚀 2. VERSION LINKEDIN (TRÈS IMPORTANT 🔥)

Voici un post **qui peut vraiment attirer recruteurs + visibilité** :

🚀 Mise en place d'un FIREWALL sous Ubuntu avec iptables 🔐

Dans le cadre de ma formation en sécurité informatique, j'ai réalisé un projet pratique consistant à déployer un firewall Linux complet avec routage et NAT.

💡 Objectifs :
✔️ Sécuriser un réseau local
✔️ Contrôler les flux entrants/sortants
✔️ Permettre l'accès Internet via NAT

🛠️ Technologies utilisées :

- Ubuntu 22.04
- iptables
- Netplan
- VMware

🔐 Implémentation :

- Politique stricte (DROP par défaut)
- Autorisation contrôlée des flux (DNS, ICMP, LAN)
- Activation du routage IP
- Mise en place du NAT (MASQUERADE)

🌐 Résultat :
✔️ Les machines du LAN accèdent à Internet
✔️ Le trafic est filtré efficacement
✔️ L'infrastructure est sécurisée

🎯 Compétences développées :

- Administration système Linux
- Sécurité réseau
- Firewalling & NAT

📌 Ce type de projet est essentiel pour comprendre les bases de la cybersécurité et des architectures réseau sécurisées.

#CyberSecurity #Linux #Networking #iptables #SysAdmin #ITSecurity #Ubuntu

---

⚡ Petit conseil (très important)

Si tu veux vraiment **passer au niveau supérieur sur LinkedIn** :

- Ajoute 📸 screenshots (iptables, ping, topologie)

- Ajoute un schéma (draw.io)
- Mets ton GitHub si possible

Si tu veux, je peux maintenant :
👉 transformer ça en **PDF ultra pro (page de garde + sommaire + design)**
👉 ou te faire un **schéma réseau propre comme un ingénieur (draw.io style)**